

Capítulo 5: Union Types y Narrowing

Los Union Types permiten crear un tipo compuesto que puede aceptar dos o más tipos diferentes. Esto resulta útil cuando una variable o parámetro puede recibir distintos tipos de datos válidos sin necesidad de utilizar el tipo `any`.

Por ejemplo, una variable definida como `string | number` puede almacenar tanto cadenas de texto como números. De esta manera, TypeScript sigue manteniendo el control de los tipos permitidos mientras ofrece mayor flexibilidad.

Cuando se trabaja con un Union Type, TypeScript solo permite utilizar propiedades o métodos que estén disponibles en todos los tipos que forman la unión. Esto evita errores que podrían ocurrir si se intentara utilizar una característica que existe en un tipo pero no en otro.

Para resolver esta limitación, TypeScript utiliza un proceso llamado Narrowing. El narrowing ocurre cuando el compilador analiza el código y determina que una variable corresponde a un tipo más específico dentro de la unión. Gracias a esto, es posible acceder a métodos y propiedades que solo existen en ese tipo concreto.

Es importante recordar que los tipos de TypeScript solo existen durante el desarrollo y desaparecen cuando el código se ejecuta. Por esta razón, no es posible consultar directamente un tipo de TypeScript en tiempo de ejecución. Sin embargo, JavaScript proporciona el operador `typeof`, que permite identificar el tipo real de un valor mientras el programa está ejecutándose y adaptar el comportamiento del código según sea necesario.

Ideas clave

- Un Union Type permite que un valor pertenezca a dos o más tipos diferentes.
- `string | number` significa que el valor puede ser una cadena de texto o un número.
- Solo pueden utilizarse métodos que existan en todos los tipos de la unión.
- El Narrowing permite que TypeScript determine un tipo más específico según el contexto.
- Los tipos de TypeScript no existen en tiempo de ejecución.
- El operador `typeof` de JavaScript permite identificar tipos durante la ejecución del programa.
- Los Union Types aportan flexibilidad sin perder la seguridad de tipos.